

Security Audit

Haiku 3rd (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	29
Conclusion	30
Our Methodology	31
Disclaimers	33
About Hashlock	34

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Haiku team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure that the project's team and community that the deployed contracts are secure.

Project Context

Haiku (accessible at haiku.trade) is an AI-powered DeFi aggregator and intent-driven trading platform that lets users execute complex, multi-step strategies—such as swaps, bridges, lending, borrowing, and liquidity management—through simple natural language commands or declarative intents. It optimizes execution using a convex optimiser for minimal slippage and swift outcomes, all bundled into a single on-chain transaction. The platform supports multiple EVM-compatible networks (e.g., Arbitrum, Base, BNB Chain) and has undergone security audits, with the most recent one in February 2025.

Additionally, Haiku incorporates a token-based system centered around the \$HKU (an ERC-20 token), which powers the ecosystem and is tied to engagement via a Karma points and airdrop program. Users earn points through activities like executing intents, building referral networks, and participating in social/community challenges—structured across tiers (e.g., Pebble to Monk)—ahead of the Token Generation Event (TGE)

Project Name: Haiku

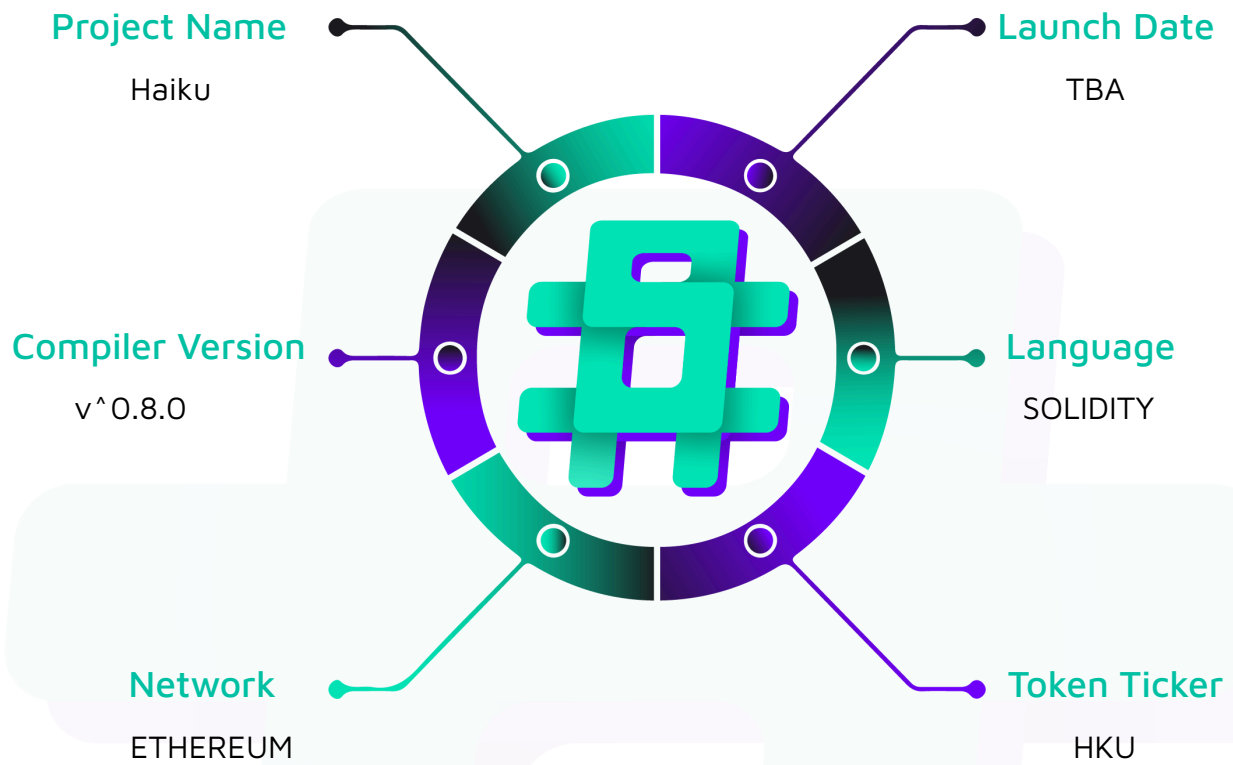
Project Type: DeFi

Compiler Version: ^0.8.0

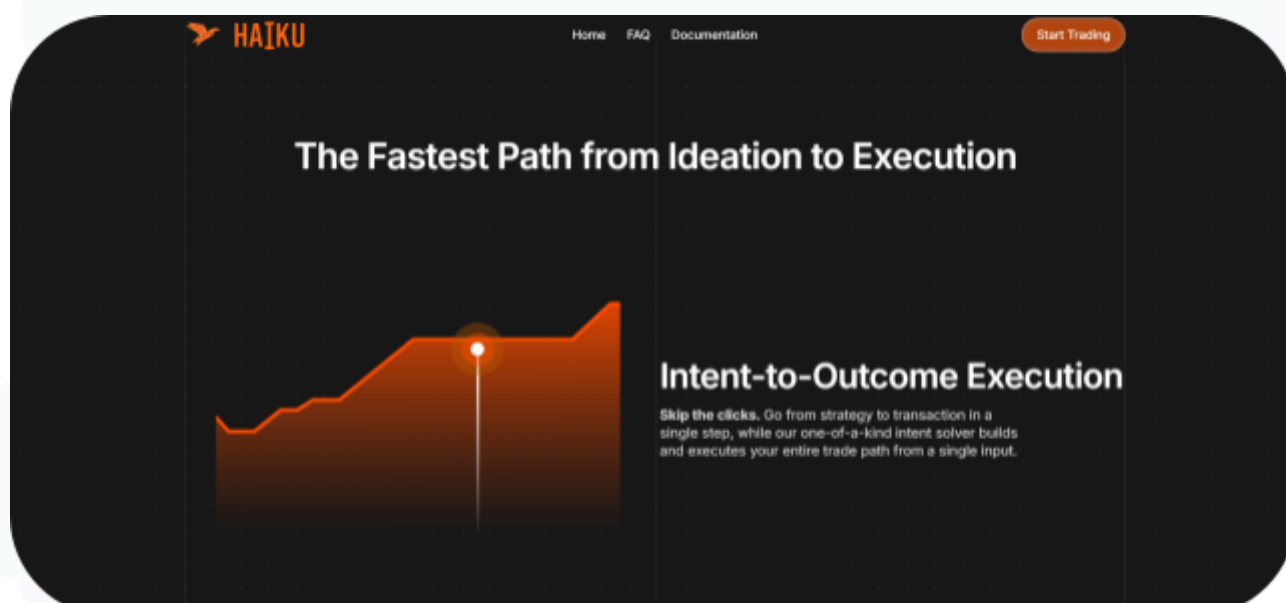
Website: <https://haiku.trade/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Haiku project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Haiku Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	February, 2026
Contract 1	UniswapV3LiquidityManager.sol
Audited GitHub Commit Hash	7972b2dcfa7ff84f74bbe17339d199f8a6a05781
Fix Review GitHub Commit Hash	1de7df68ae82ce20dbcb590a8b27205d73221522

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

2 Medium-severity vulnerabilities

4 Low-severity vulnerabilities

1 Gas Optimisation

3 QAs

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
INonfungiblePositionManager.sol - Allows users to: - Library/Interface — callable helpers only	Contract achieves this functionality.
IUniswapV3Pool.sol - Allows users to: - Library/Interface — callable helpers only	Contract achieves this functionality.
UniswapV3LiquidityManager.sol - Allows users to: - Add liquidity with custom ticks and key - Remove liquidity for the owned unique key - Read tracked NFT positions and current tick - Send ERC721 tokens to this contract - Allows admins to: - Transfer managed NFT to the contract owner - Update whitelist entries for accounts - Update default tick range width - Withdraw ERC20 tokens via the emergency path	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Haiku project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Haiku project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] UniswapV3LiquidityManager#addLiquidity - Missing Slippage Protection Exposes Users to Sandwich Attacks

Description

`addLiquidity` and `removeLiquidity` both pass `amount0Min: 0` and `amount1Min: 0` when calling the Uniswap V3 `NonfungiblePositionManager`. This means the contract accepts any resulting token ratio, however unfavourable, without reverting. A sandwich attacker can manipulate the pool price before the transaction and restore it afterward, extracting value from the liquidity provider at zero risk.

Vulnerability Details

In `addLiquidity` and `removeLiquidity`, the slippage parameters are hardcoded to zero:

```
// addLiquidity
(tokenId, liquidity, vars.amount0Used, vars.amount1Used) = positionManager.mint(
    INonfungiblePositionManager.MintParams({
        ...
        amount0Min: 0,    // hardcoded zero - no slippage protection
        amount1Min: 0,    // hardcoded zero - no slippage protection
        recipient: address(this),
        deadline: block.timestamp
    })
);

// removeLiquidity - L348
INonfungiblePositionManager.DecreaseLiquidityParams memory decreaseParams =
INonfungiblePositionManager
    .DecreaseLiquidityParams({
        tokenId: tokenId,
        liquidity: liquidity,
        amount0Min: 0,    // hardcoded zero - no slippage protection
```

```

    amount1Min: 0,    // hardcoded zero - no slippage protection
    deadline: block.timestamp
  });

```

Because `addLiquidity` accepts caller-supplied `amountA` / `amountB` and transfers them from `msg.sender`, the caller is the natural target. An MEV bot can front-run the transaction to shift the pool price, causing the position manager to consume far less of one token (the remainder is refunded), while the minted position is severely skewed and worth less at the fair market price. The back-run restores the price, and the attacker profits from the price impact.

Impact

Any caller of `addLiquidity` or `removeLiquidity` is exposed to sandwich attacks from MEV bots. Because `addLiquidity` is permissionless (anyone can call it), a user depositing a significant amount can lose an arbitrary fraction of their intended deposit value in a single block. For `removeLiquidity`, the keyOwner receives fewer tokens than the position is worth at the current fair price. The loss is bounded only by available MEV bot capital and pool depth, with no protocol-level protection for the user.

Recommendation

Accept caller-specified `amount0Min` and `amount1Min` parameters for both `addLiquidity` and `removeLiquidity`, and forward them directly to the position manager.

```

function addLiquidity(
    address poolAddress,
    uint256 amountA,
    uint256 amountB,
    int24 tickLower,
    int24 tickUpper,
+   uint256 amount0Min,
+   uint256 amount1Min,
    bytes32 uniqueKey
) external returns (uint256 tokenId, uint128 liquidity) {
    ...
    (tokenId, liquidity, vars.amount0Used, vars.amount1Used) = positionManager.mint(
        INonfungiblePositionManager.MintParams({
            ...

```

```
-         amount0Min: 0,  
-         amount1Min: 0,  
+         amount0Min: amount0Min,  
+         amount1Min: amount1Min,  
          recipient: address(this),  
          deadline: block.timestamp  
      })  
  );
```

Apply the same change to `removeLiquidity`.

Status

Resolved

[M-02] UniswapV3LiquidityManager#addLiquidity - Front-Run Key Squatting Enables UniqueKey DoS

Description

`addLiquidity` is permissionless and accepts an arbitrary caller-provided `uniqueKey`, but uniqueness is enforced globally through `keyToNFT[uniqueKey] != 0`. This allows a mempool observer to front-run a victim transaction and register the same key first with minimal liquidity. The victim transaction then reverts with `DuplicateKey`, even though the victim intended to use that key legitimately.

Vulnerability Details

The contract treats `uniqueKey` as a one-time global namespace without binding it to the expected owner, signer, or an off-chain authorization proof. In `addLiquidity`, the only uniqueness check is:

```
if (keyToNFT[uniqueKey] != 0) revert DuplicateKey();  
...  
keyToNFT[uniqueKey] = tokenId;  
keyToOwner[uniqueKey] = msg.sender;
```

Because any address can call `addLiquidity`, an attacker can copy a pending victim transaction's `uniqueKey`, submit a higher-priority transaction with tiny amounts, and permanently occupy that key. If the attacker never calls `removeLiquidity`, the key remains blocked indefinitely. This is a practical denial-of-service vector when `uniqueKey` is used as an external order/intent identifier.

Proof of Concept

```
function testUniqueKeyFrontRunSquatting() public {  
    // --- Setup and Preconditions ---  
    address victim = makeAddr("victim");  
    address attacker = makeAddr("attacker");  
    bytes32 key = keccak256("victim-position-1");  
    token0.mint(victim, 1_000e18);  
    token1.mint(victim, 1_000e18);  
    token0.mint(attacker, 1);  
    token1.mint(attacker, 1);
```



```

// --- Exploit Execution ---
vm.startPrank(attacker);
token0.approve(address(manager), type(uint256).max);
token1.approve(address(manager), type(uint256).max);
manager.addLiquidity(address(pool), 1, 1, -10, 10, key);
vm.stopPrank();

vm.startPrank(victim);
token0.approve(address(manager), type(uint256).max);
token1.approve(address(manager), type(uint256).max);
vm.expectRevert(UniswapV3LiquidityManager.DuplicateKey.selector);
manager.addLiquidity(address(pool), 1_000e18, 1_000e18, -10, 10, key);
vm.stopPrank();

// --- Post-condition Check ---
assertEq(manager.keyToOwner(key), attacker); // PoC succeeds if attacker permanently
controls the key mapping
}

```

Impact

An attacker can cheaply block a target `uniqueKey` before the intended transaction executes, causing deterministic transaction failure and forcing retries with new keys. If integrators map `uniqueKey` to a canonical order/intent ID, this becomes a protocol-level liveness failure for that order flow rather than a one-time nuisance, with repeated gas loss for victims and potentially permanent inability to execute specific intents.

Recommendation

Bind `uniqueKey` to an authenticated owner context instead of accepting unrestricted global claims. A minimal hardening is to reserve keys per owner, or require an owner signature over `(uniqueKey, pool, tick range, amount bounds, nonce, deadline)` before first registration.

```

- if (keyToNFT[uniqueKey] != 0) revert DuplicateKey();
+ bytes32 scopedKey = keccak256(abi.encode(msg.sender, uniqueKey));
+ if (keyToNFT[scopedKey] != 0) revert DuplicateKey();
...

```

```
- keyToNFT[uniqueKey] = tokenId;  
- keyToOwner[uniqueKey] = msg.sender;  
+ keyToNFT[scopedKey] = tokenId;  
+ keyToOwner[scopedKey] = msg.sender;
```

If the key must remain globally unique for business logic, replace caller-claimed registration with signature-based authorization from the party that owns the off-chain intent identifier.

Status

Resolved

Low

[L-01] UniswapV3LiquidityManager#transferNFTToOwner - Stale Key Mappings After NFT Withdrawal

Description

When `transferNFTToOwner` is called, the function sets `ownedNFTs[tokenId] = false` and removes the `tokenId` from the `nftPositions` array, but does not clear `keyToNFT[uniqueKey]` or `keyToOwner[uniqueKey]`. These mappings are left pointing to a `tokenId` that is no longer tracked by the contract. Any off-chain indexer or integration relying on `keyToNFT` / `keyToOwner` to reflect the current ownership state will read stale data after this call. Additionally, the `uniqueKey` cannot be reused for a new position because `keyToNFT[uniqueKey] != 0` causes `addLiquidity` to revert with `DuplicateKey`.

Recommendation

Maintain a reverse mapping `nftToKey` from `tokenId` to `uniqueKey` and use it inside `transferNFTToOwner` to delete both key mappings atomically at the time of NFT withdrawal.

```
+ mapping(uint256 => bytes32) public nftToKey;

// In addLiquidity, after keyToNFT assignment:
+ nftToKey[tokenId] = uniqueKey;

function transferNFTToOwner(uint256 tokenId) external onlyOwner {
    if (tokenId == 0 || !ownedNFTs[tokenId]) revert NFTNotOwned();
    ownedNFTs[tokenId] = false;
    _removeNFTFromArray(tokenId);
+   bytes32 key = nftToKey[tokenId];
+   if (key != bytes32(0)) {
+       delete keyToNFT[key];
+       delete keyToOwner[key];
+       delete nftToKey[tokenId];
+   }
```

```
positionManager.safeTransferFrom(address(this), owner(), tokenId);  
emit NFTTransferred(tokenId, owner());  
}
```

Also, clean up `nftToKey` inside `removeLiquidity` to keep the reverse mapping consistent.

Status

Resolved

[L-02] UniswapV3LiquidityManager#removeLiquidity - CEI Pattern Violation

Description

`removeLiquidity` performs all external calls (`decreaseLiquidity`, `collect`, `safeTransfer` for both tokens) before updating the contract's internal state (`ownedNFTs[tokenId] = false`, `delete keyToNFT`, `delete keyToOwner`). This violates the Checks-Effects-Interactions (CEI) pattern. Although no profitable reentrancy exploit path exists in the current implementation — because `decreaseLiquidity` with `liquidity = 0` reverts in the Uniswap V3 NonfungiblePositionManager (`require(params.liquidity > 0)`), and the `onERC721Received` callback fires after state cleanup — the pattern itself is fragile. If the pool interaction is replaced or augmented in the future, or if a non-standard position manager is used, the stale state window could become exploitable.

Recommendation

Move all state-mutating operations (`ownedNFTs[tokenId] = false`, `delete keyToNFT`, `delete keyToOwner`, `_removeNFTFromArray`) before the external token transfer calls, following the CEI pattern. Alternatively, add a `nonReentrant` modifier from OpenZeppelin.

```
+ import {ReentrancyGuard} from
'lib/openzeppelin-contracts/contracts/security/ReentrancyGuard.sol';

- contract UniswapV3LiquidityManager is Ownable2Step, IERC721Receiver {
+ contract UniswapV3LiquidityManager is Ownable2Step, IERC721Receiver, ReentrancyGuard
{

- function removeLiquidity(bytes32 uniqueKey) external {
+ function removeLiquidity(bytes32 uniqueKey) external nonReentrant {
```

Status

Resolved

[L-03] UniswapV3LiquidityManager - deadline: block.timestamp Provides No Protection

Description

Both `addLiquidity` and `removeLiquidity` pass `deadline: block.timestamp` to the Uniswap V3 `NonfungiblePositionManager`. Because `block.timestamp` equals the timestamp of the block being processed, the deadline check inside the position manager is always satisfied regardless of when the transaction is actually included. This effectively disables the deadline mechanism, allowing validators to hold and include transactions at an arbitrarily unfavourable price without triggering a revert.

Recommendation

Accept a caller-supplied `deadline` parameter for both functions and forward it to the position manager.

```
function addLiquidity(
    address poolAddress, uint256 amountA, uint256 amountB,
-   int24 tickLower, int24 tickUpper, bytes32 uniqueKey
+   int24 tickLower, int24 tickUpper, bytes32 uniqueKey, uint256 deadline
) external returns (uint256 tokenId, uint128 liquidity) {
    ...
-   deadline: block.timestamp
+   deadline: deadline
```

Apply the same change to `removeLiquidity`.

Status

Resolved

[L-04] UniswapV3LiquidityManager#setTickRangeWidth - Missing Upper Bound Allows Overflow Panic

Description

`setTickRangeWidth` accepts any `uint256` with no upper bound check. Inside `_calculateTickRange`, the value is cast as `int24(int256(defaultTickRangeWidth)) * tickSpacing`. If `defaultTickRangeWidth` is large enough that the result exceeds `type(int24).max` (8,388,607), Solidity 0.8's checked arithmetic panics and reverts, bricking any function that calls `_calculateTickRange`.

Recommendation

Add a maximum bound check before updating the state variable.

```
function setTickRangeWidth(uint256 newWidth) external onlyOwner {  
+   require(newWidth <= 887272, "width exceeds max tick range");  
    uint256 oldWidth = defaultTickRangeWidth;  
    defaultTickRangeWidth = newWidth;  
    emit TickRangeWidthUpdated(oldWidth, newWidth);  
}
```

Status

Resolved

QA

[Q-01] UniswapV3LiquidityManager - Dead Code: `_calculateTickRange`, `defaultTickRangeWidth`, `setTickRangeWidth`

Description

`_calculateTickRange` is an `internal` function that is never called by any function in the contract. `addLiquidity` accepts explicit `tickLower` / `tickUpper` parameters from the caller and does not invoke `_calculateTickRange`. As a result, the `defaultTickRangeWidth` state variable, its setter `setTickRangeWidth`, and the `TickRangeWidthUpdated` event are all dead code that deploy with the contract and waste gas on any future setter calls with no effect on behavior.

Recommendation

Either wire `_calculateTickRange` into `addLiquidity` as intended, or remove the function `defaultTickRangeWidth`, `setTickRangeWidth`, and `TickRangeWidthUpdated` entirely.

Status

Resolved

[Q-02] UniswapV3LiquidityManager - Dead Code: `onlyAuthorized` Modifier and `whitelist` Mapping

Description

The `onlyAuthorized` modifier and `whitelist` mapping are defined but the modifier is never applied to any external or public function. `setWhitelist` updates the mapping, but no function gate-checks against it, so whitelist entries have no effect on contract behavior. Callers and integrators may incorrectly assume that whitelisted addresses have capabilities they do not actually possess. The `NotAuthorized` error is also unreachable through any current code path.

Recommendation

Apply `onlyAuthorized` to the functions intended to be access-controlled (e.g., `addLiquidity`), or remove the modifier, `whitelist` mapping, `setWhitelist`, `WhitelistUpdated` event, and `NotAuthorized` error entirely.

Status

Resolved

[Q-03] UniswapV3LiquidityManager - Unused Declarations: LiquidityMoved Event and InvalidTokenId Error

Description

The `LiquidityMoved` event and `InvalidTokenId` error are declared but never emitted or used anywhere in the contract. They marginally increase bytecode size and create misleading signals for off-chain indexers that may subscribe to events expecting them to fire.

Recommendation

Remove `LiquidityMoved` and `InvalidTokenId`, or implement the functionality that was intended to use them.

Status

Resolved

Gas

[G-01] UniswapV3LiquidityManager#_removeNFTFromArray - O(n) Linear Scan on Every Removal

Description

`\_removeNFTFromArray` iterates the entire `nftPositions` array to find the target `tokenId`. Both `removeLiquidity` and `transferNFTToOwner` call this function on every execution. Gas cost scales linearly with the number of tracked positions, making removals increasingly expensive as the protocol accumulates positions.

Recommendation

Maintain a companion `nftPositionIndex` mapping from `tokenId` to its array index for O(1) lookup and removal.

```
+ mapping(uint256 => uint256) private nftPositionIndex;

// In addLiquidity, after nftPositions.push(tokenId):
+ nftPositionIndex[tokenId] = nftPositions.length - 1;

function _removeNFTFromArray(uint256 tokenId) internal {
-     uint256 length = nftPositions.length;
-     for (uint256 i = 0; i < length; i++) {
-         if (nftPositions[i] == tokenId) {
-             nftPositions[i] = nftPositions[length - 1];
-             nftPositions.pop();
-             break;
-         }
-     }
+     uint256 index = nftPositionIndex[tokenId];
+     uint256 last = nftPositions[nftPositions.length - 1];
+     nftPositions[index] = last;
+     nftPositionIndex[last] = index;
+     nftPositions.pop();
+     delete nftPositionIndex[tokenId];
}
```

Status

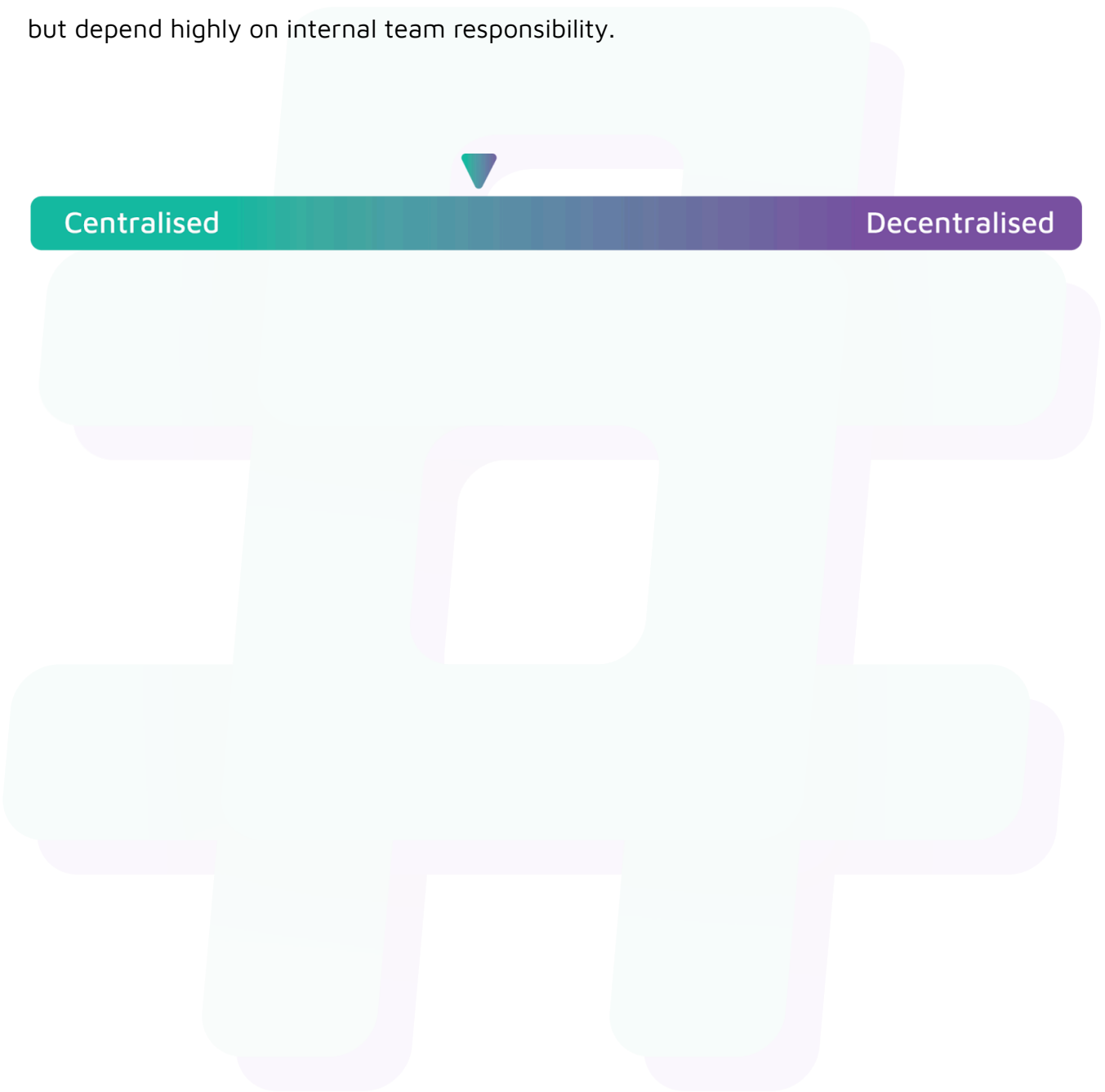
Resolved



Centralisation

The Haiku project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Haiku project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.